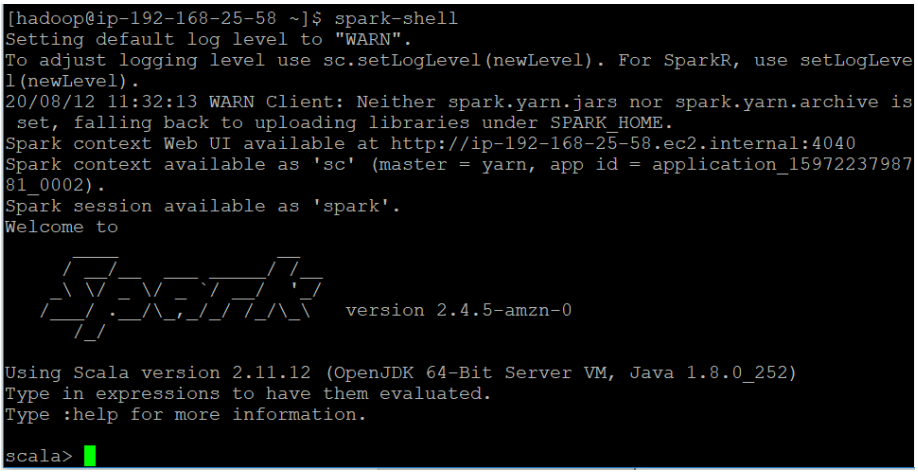
COSC2637/2633 Big Data Processing

Scala Practices



https://www.dataneb.com/post/spark-transformations-examples-in-scala
https://www.dataneb.com/post/spark-transformations-examples-scala
https://www.dataneb.com/post/spark-rdd-transformations-and-actions-example

- **Anonymous Function (Lambda Function)**

```
scala> val lfunc = (x:Int) => x + x

scala> lfunc(3)
```

```
scala> val lfunc=(x:Int)=>x+x
lfunc: Int => Int = <function1>

scala> lfunc(3)
res0: Int = 6
```

- **Access a text file**

```
scala> val pigs = sc.textFile("hdfs:///user/ke/3littlepigs")
```

A HDFS file /user/ke/3littlepigs

- **Spark Transformations – map(*func*)**

```
scala> val words = pigs.map(x => x.split(" "))

scala> words.collect()
```

Don't worry about collect() action, it's very basic Spark action which is used to return all the elements.

```
scala> val pigs = sc.textFile("hdfs:///user/ke/3littlepigs")
pigs: org.apache.spark.rdd.RDD[String] = hdfs:///user/ke/3littlepigs MapPartitio
nsRDD[42] at textFile at <console>:24

scala> val words = pigs.map(x=>x.split(" "))
words: org.apache.spark.rdd.RDD[Array[String]] = MapPartitionsRDD[43] at map at
<console>:25

scala> words.collect()
res38: Array[Array[String]] = Array(Array("", "", "", "", "", "", "", "", "", ""
, "", "", "", "", "", "", "", "", THE, THREE, LITTLE, PIGS), Array(""), Array(""
, "", "", Once, upon, a, time, ., ., ., there, were, three, little, pigs,, who,
left, their, mummy), Array(and, daddy, to, see, the, world.), Array("", "", "",
All, summer, long,, they, roamed, through, the, woods, and, over, the, plains,pl
aying), Array(games, and, having, fun., None, were, happier, than, the, three, l
ittle, pigs,, and, they), Array(easily, made, friends, with, everyone., Wherever
, they, went,, they, were, given, a, warm), Array(welcome,, but, as, summer, dre
w, to, a, close,, they, realized, that, folk, were, drifting), Array(back, to, t
heir, usual, jobs,, and, preparing, for, winter., Autumn, came, and, it, bega...
scala>
```

```
scala> pigs.map(x => x.split(" ").length).collect()
```

```
scala> pigs.map(x=>x.split(" ").length).collect()
res9: Array[Int] = Array(22, 1, 19, 6, 15, 14, 13, 14, 14, 15, 17, 16, 14, 6, 14
, 15, 14, 1, 17, 9, 17, 16, 12, 17, 13, 17, 6, 18, 16, 3, 19, 1, 15, 7, 17, 17,
16, 16, 15, 15, 13, 12, 12, 18, 12, 18, 2, 17, 12, 14, 18, 14, 13, 6, 16, 14, 13
, 15, 16, 17, 14, 15, 15, 11, 16, 15, 16, 1, 18, 15, 3, 16, 17, 18, 16, 1, 13, 1
6, 17, 14, 14, 2, 15, 12)
```

```
scala> pigs.map(x => x.split(" ").size).collect()
```

```
scala> pigs.map(x=>x.split(" ").size).collect()
res10: Array[Int] = Array(22, 1, 19, 6, 15, 14, 13, 14, 14, 15, 17, 16, 14, 6, 1
4, 15, 14, 1, 17, 9, 17, 16, 12, 17, 13, 17, 6, 18, 16, 3, 19, 1, 15, 7, 17, 17,
 16, 16, 15, 15, 13, 12, 12, 18, 12, 18, 2, 17, 12, 14, 18, 14, 13, 6, 16, 14, 1
3, 15, 16, 17, 14, 15, 15, 11, 16, 15, 16, 1, 18, 15, 3, 16, 17, 18, 16, 1, 13,
16, 17, 14, 14, 2, 15, 12)
```

```
scala> pigs.map(x => x.length).collect()
```

```
scala> pigs.map(x=>x.length).collect()
res12: Array[Int] = Array(39, 0, 76, 27, 77, 76, 78, 77, 76, 77, 78, 75, 77, 28,
 60, 78, 73, 5, 77, 47, 76, 74, 59, 75, 74, 70, 39, 77, 76, 22, 74, 14, 71, 40,
77, 72, 74, 78, 78, 77, 72, 76, 63, 77, 69, 78, 10, 78, 59, 51, 76, 77, 72, 31,
77, 70, 72, 77, 77, 75, 78, 73, 73, 58, 75, 76, 76, 9, 75, 79, 12, 72, 74, 78, 7
5, 5, 73, 74, 77, 73, 76, 9, 68, 46)
```

```
scala> pigs.map(x => x.toUpperCase()).collect()
```

```
scala> pigs.map(x=>x.toUpperCase).collect()
res13: Array[String] = Array("                    THE THREE LITTLE PIGS", "", "
ONCE UPON A TIME . . . THERE WERE THREE LITTLE PIGS, WHO LEFT THEIR MUMMY", AND
DADDY TO SEE THE WORLD., "    ALL SUMMER LONG, THEY ROAMED THROUGH THE WOODS AND
OVER THE PLAINS,PLAYING", GAMES AND HAVING FUN. NONE WERE HAPPIER THAN THE THREE
 LITTLE PIGS, AND THEY, "EASILY MADE FRIENDS WITH EVERYONE. WHEREVER THEY WENT,
THEY WERE GIVEN A WARM ", WELCOME, BUT AS SUMMER DREW TO A CLOSE, THEY REALIZED
THAT FOLK WERE DRIFTING, BACK TO THEIR USUAL JOBS, AND PREPARING FOR WINTER. AUT
UMN CAME AND IT BEGAN, TO RAIN. THE THREE LITTLE PIGS STARTED TO FEEL THEY NEEDE
D A REAL HOME. SADLY, THEY KNEW THAT THE FUN WAS OVER NOW AND THEY MUST SET TO W
ORK LIKE THE OTHERS,, OR THEY'D BE LEFT IN THE COLD AND RAIN, WITH NO ROOF OV...
scala>
```

- **Spark Transformations – flatmap(*func*)**

```
scala> val rdd = sc.parallelize(Seq("Where is the 3 pigs", "fairy land "))

scala> rdd.collect
```

```
scala> val rdd = sc.parallelize(Seq("Where is the 3 pigs", "fairy lang"))
rdd: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[22] at parallelize
 at <console>:24

scala> rdd.collect
res14: Array[String] = Array(Where is the 3 pigs, fairy lang)
```

```
scala> rdd.map(x => x.split(" ")).collect
```

```
scala> rdd.map(x=>x.split(" ")).collect
res15: Array[Array[String]] = Array(Array(Where, is, the, 3, pigs), Array(fairy,
 lang))

scala>
```

```
scala> rdd.flatMap(x => x.split(" ")).collect
```

```
scala> rdd.flatMap(x=>x.split(" ")).collect
res18: Array[String] = Array(Where, is, the, 3, pigs, fairy, lang)

scala>
```

When map() transformation is applied, it returned array of array string Array[Array[String]]. It has basically two separate array of strings within an array. So for each element we got one output (1st element => 1 element (Where, is, the, 3, pigs), 2nd element => 1 element (fairy land)). And those individual elements are collection of words separated by comma.

But if you see flatMap(), output is flattened to single array of string Array[String].

```
scala> rdd.map(x => x.split(" ")).count()
```

```
scala> rdd.map(x=>x.split(" ")).count()
res19: Long = 2
```

```
scala> rdd.flatMap(x => x.split(" ")).count()
```

```
scala> rdd.flatMap(x=>x.split(" ")).count()
res20: Long = 7
```

- **Spark Transformations – filter(*func*)**

```
scala> rdd.filter(x=>x.contains("pigs")).collect
```

```
scala> rdd.collect
res29: Array[String] = Array(Where is the 3 pigs, fairy lang)

scala> rdd.filter(x=>x.contains("pigs")).collect
res30: Array[String] = Array(Where is the 3 pigs)

scala>
```

```
scala> rdd.filter(x=>x.contains("g")).collect
```

```
scala> rdd.filter(x=>x.contains("g")).collect
res32: Array[String] = Array(Where is the 3 pigs, fairy lang)
```

```
scala> rdd.filter(x=>x.toLowerCase.contains("pig")).collect
```

```
scala> rdd.filter(x=>x.contains("G")).collect
res33: Array[String] = Array()
```

```
scala> rdd.filter(x=>x.toUpperCase.contains("G")).collect
res34: Array[String] = Array(Where is the 3 pigs, fairy lang)
```

- **Spark Transformations – mapPartitions(*func*)**

```
def parallelize[T](seq: Seq[T], numSlices: Int = defaultParallelism)
(implicit arg0: ClassTag[T]): RDD[T]
```

mapPartitions(*func*)

Similar to map, but runs separately on each partition (block) of the RDD, so *func* must be of type Iterator<T> => Iterator<U> when running on an RDD of type T.

Similar to map() transformation but in this case function runs separately on each partition (block) of RDD unlike map() where it was running on each element. Hence mapPartitions are also useful when you are looking for performance gain (calls your function once/partition not once/element).

- *Suppose you have elements from 1 to 100 distributed among 10 partitions, i.e. 10 elements/partition. map() transformation will call func 100 times to process these 100 elements but in case of mapPartitions(), func will be called once/partition i.e. 10 times.*
- *Secondly, mapPartitions() holds the data in-memory, i.e., it will store the result in memory until all the elements of the partition has been processed.*
  - *mapPartitions() will return the result only after it finishes processing of whole partition.*
- *mapPartitions() requires an iterator input unlike map() transformation.*

An iterator is a way to access collection of elements one-by-one, its similar to collection of elements like List(), Array(), Dict(), etc; but the difference is that iterator doesn't load the whole collection of elements in memory at together. Instead iterator loads elements one after another. In Scala you access these elements with hasNext *and* Next *operation.*

```
scala> sc.parallelize(1 to 9, 3).map(x=>(x, "Hello")).collect
```

In this example, I have applied map() transformation on dataset distributed between 3 partitions so that you can see function is called 9 times.

```
scala> sc.parallelize(1 to 9, 3).map(x=>(x, "Hello")).collect
res49: Array[(Int, String)] = Array((1,Hello), (2,Hello), (3,Hello), (4,Hello),
(5,Hello), (6,Hello), (7,Hello), (8,Hello), (9,Hello))
```

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(Array("Hello").iterator)).collect
```

In this example, when we applied mapPartitions(), you will notice it ran 3 times, i.e., for each partition once. We had to convert string "Hello" into iterator because mapPartitions() takes iterator.

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(Array("Hello").iterator)).collect
res51: Array[String] = Array(Hello, Hello, Hello)
```

```
scala> sc.parallelize(1 to 9, 5).mapPartitions(x=>(Array("Hello").iterator)).collect
res52: Array[String] = Array(Hello, Hello, Hello, Hello, Hello)
```

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next).iterator)).collect
```

In this step, we tried to get the iterator value to showcase the dataset. Note that next *is always increasing value, so you can't step back.*

```
scala> sc.parallelize(1 to 9,3 ).mapPartitions(x=>(List(x.next).iterator)).colle
ct
res48: Array[Int] = Array(1, 4, 7)
```

```
scala> sc.parallelize(1 to 9,5 ).mapPartitions(x=>(List(x.next).iterator)).colle
ct
res54: Array[Int] = Array(1, 2, 4, 6, 8)
```

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next,x.next,
"|").iterator)).collect
```

In first call next value for partition 1 changed from 1 => 1, 2, for partition 2 it changed from 4 => 4, 5 and similarly for partition 3 it changed from 7 => 7, 8.

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next, x.next, "|").ite
rator)).collect
res70: Array[Any] = Array(1, 2, |, 4, 5, |, 7, 8, |)
```

You can keep this increasing until hasNext is False (hasNext is a property of iteration which tells you whether collection has ended or not, it returns you True or False based on items left in the collection). For example,

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next,
x.hasNext).iterator)).collect
```

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next, x.hasNext).itera
tor)).collect
res72: Array[AnyVal] = Array(1, true, 4, true, 7, true)
```

You can see hasNext is true because there are elements left in each partition. Now suppose we access all three elements from each partition, then hasNext will result false. For example,

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next, x.next,
x.next, x.hasNext).iterator)).collect
```

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next, x.next, x.next,
x.hasNext).iterator)).collect
res74: Array[AnyVal] = Array(1, 2, 3, false, 4, 5, 6, false, 7, 8, 9, false)
```

If accessing next 4th time, you will get error.

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next, x.next,
x.next, x.next, x.hasNext).iterator)).collect
```

```
scala> sc.parallelize(1 to 9, 3).mapPartitions(x=>(List(x.next, x.next, x.next,
x.next, x.hasNext).iterator)).collect
20/08/13 04:29:33 WARN TaskSetManager: Lost task 0.0 in stage 51.0 (TID 149, ip-
192-168-25-9.ec2.internal, executor 32): java.util.NoSuchElementException: next
on empty iterator
        at scala.collection.Iterator$$anon$2.next(Iterator.scala:39)
```

- **Spark Transformations – mapPartitionsWithIndex(func)**

```
scala> val mp = sc.parallelize(List("One","Two","Three","Four",
"Five","Six","Seven","Eight","Nine"), 3)

scala> mp.collect
```

```
scala> val mp = sc.parallelize(List("one", "two", "three", "four", "five", "six"
, "seven", "eight", "nine"), 3)
mp: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[76] at parallelize
at <console>:24

scala> mp.collect
res55: Array[String] = Array(one, two, three, four, five, six, seven, eight, nin
e)
```

```
scala> mp.mapPartitionsWithIndex((index, iterator) =>
{iterator.toList.map(x => x + "=>" + index ).iterator} ).collect
```

```
scala> mp.mapPartitionsWithIndex((index, iterator)=>{iterator.toList.map(x=>x+"=
>"+index).iterator}).collect
res56: Array[String] = Array(one=>0, two=>0, three=>0, four=>1, five=>1, six=>1,
 seven=>2, eight=>2, nine=>2)
```

- **Spark Transformations – sample()**

> **sample**(*withReplacement*, *fraction*, *seed*)
>
> Sample a fraction *fraction* of the data, with or without replacement, using a given random number generator seed.
>
> Note that it doesn't guarantee to provide the exact number of the fraction of records. The fraction argument doesn't represent the fraction of the actual size of the RDD. It represents the probability of each element in the population getting selected for the sample.

```
scala> sc.parallelize(1 to 10).sample(true, .4).collect
```

```
scala> sc.parallelize(1 to 10).sample(true, .4).collect
res58: Array[Int] = Array(1, 1, 5, 6, 8)
```

```
scala> sc.parallelize(1 to 10).sample(true, .2).collect
```

```
scala> sc.parallelize(1 to 10).sample(true, .2).collect
res59: Array[Int] = Array(2, 7, 10)
```

```
scala> sc.parallelize(1 to 10).sample(false, .5).collect
```

```
scala> sc.parallelize(1 to 10).sample(false, .5).collect
res64: Array[Int] = Array(3, 4, 5, 6, 7, 10)
```

- **Spark Transformations – union()**

```
scala> val rdd1 =
sc.parallelize(List("apple","orange","grapes","mango","orange"))

scala> val rdd2 = sc.parallelize(List("red","green","yellow"))

scala> rdd1.union(rdd2).collect
```

```
scala> val rdd1 = sc.parallelize(List("apple","orange","grapes","mango","orange"
))
rdd1: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[115] at paralleli
ze at <console>:24

scala> val rdd2 = sc.parallelize(List("red","green","yellow"))
rdd2: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[116] at paralleli
ze at <console>:24

scala> rdd1.union(rdd2).collect
res76: Array[String] = Array(apple, orange, grapes, mango, orange, red, green, y
ellow)
```

```
scala> rdd2.union(rdd1).collect
```

```
scala> rdd2.union(rdd1).collect
res77: Array[String] = Array(red, green, yellow, apple, orange, grapes, mango, o
range)
```

- **Spark Transformations – intersection()**

```
scala> val rdd1 = sc.parallelize(-5 to 5)

scala> val rdd2 = sc.parallelize(1 to 10)

scala> rdd1.intersection(rdd2).collect
```

```
scala> val rdd1 = sc.parallelize(-5 to 5)
rdd1: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[120] at parallelize
at <console>:24

scala> val rdd2 = sc.parallelize(1 to 10)
rdd2: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[121] at parallelize
at <console>:24

scala> rdd1.intersection(rdd2).collect
res78: Array[Int] = Array(4, 1, 5, 2, 3)
```

- **Spark Transformations - distinct()**

```
scala> val rdd =
sc.parallelize(List("apple","orange","grapes","mango","orange"))

scala> rdd.distinct.collect
```

```
scala> val rdd = sc.parallelize(List("apple","orange","grapes","mango","orange")
)
rdd: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[128] at paralleliz
e at <console>:24

scala> rdd.distinct.collect
res79: Array[String] = Array(grapes, orange, apple, mango)
```

- **Spark Transformations - groupByKey()**

groupByKey([*numPartitions*])

When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using `reduceByKey` or `aggregateByKey` will yield much better performance.
Note: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional `numPartitions` argument to set a different number of tasks.

```
scala> val rdd = sc.parallelize(List("Hello Hello Spark Apache Hello
Dataneb Dataneb Dataneb Spark"))

scala> rdd.collect
```

```
scala> val rdd = sc.parallelize(List("Hello Hello Spark Apache Hello Dataneb Dat
aneb Dataneb Spark"))
rdd: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[0] at parallelize
at <console>:24

scala> rdd.collect()
res0: Array[String] = Array(Hello Hello Spark Apache Hello Dataneb Dataneb Datan
eb Spark)
```

```
scala> val keyValue = rdd.flatMap(words => words.split(" ")).map(x=>(x,1))
// Splitting the array and creating (K, V) pair

scala> keyValue.groupByKey.collect
```

```
scala> val keyValue = rdd.flatMap(words => words.split(" ")).map(x=>(x,1))
keyValue: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[2] at map a
t <console>:25

scala> keyValue.groupByKey.collect
res1: Array[(String, Iterable[Int])] = Array((Hello,CompactBuffer(1, 1, 1)), (Ap
ache,CompactBuffer(1)), (Spark,CompactBuffer(1, 1)), (Dataneb,CompactBuffer(1, 1
, 1)))
```

- **Spark Transformations - reduceByKey()**

reduceByKey(*func*, [*numPartitions*])

When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function *func*, which must be of type (V,V) => V. Like in `groupByKey`, the number of reduce tasks is configurable through an optional second argument.

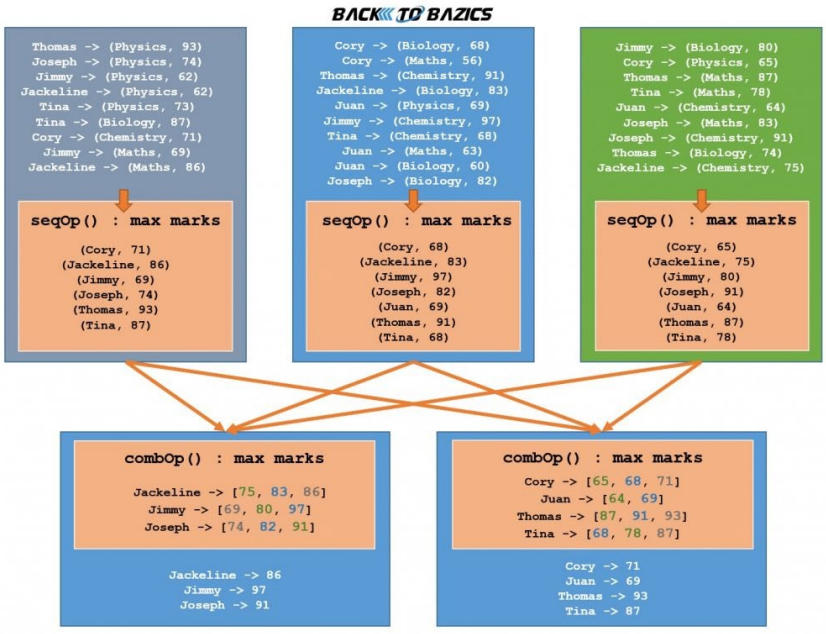
```
scala> rdd.flatMap(words => words.split(" ")).map(x=>(x,1)).reduceByKey((x, y)=>x+y).collect
```

```
scala> rdd.flatMap(words => words.split(" ")).map(x=>(x,1)).reduceByKey((x, y)=>x+y).collect
res7: Array[(String, Int)] = Array((Hello,3), (Apache,1), (Spark,2), (Dataneb,3))
```

- **Spark Transformations - aggregateByKey()**

aggregateByKey(*zeroValue*)(*seqOp*, *combOp*, [*numPartitions*])

- Second argument `seqOp` is an operation which aggregates all values of a single partition
- Third argument `combOp` is similar as `seqOp` and which further aggregates all aggregated values from different partitions



Source: https://backtobazics.com/big-data/spark/apache-spark-aggregatebykey-example/

```
scala> rdd.flatMap(words => words.split("
")).map(x=>(x,1)).aggregateByKey(0)((x, y)=> x+y, (k, v)=> k+v ).collect
```

```
scala> rdd.flatMap(words => words.split(" ")).map(x=>(x,1)).aggregateByKey(0)((x
, y)=> x+y, (k, v)=> k+v ).collect
res8: Array[(String, Int)] = Array((Hello,3), (Apache,1), (Spark,2), (Dataneb,3)
)

scala> rdd.flatMap(words => words.split(" ")).map(x=>(x,1)).aggregateByKey(2)((x
, y)=> x+y, (k, v)=> k+v ).collect
res9: Array[(String, Int)] = Array((Hello,5), (Apache,3), (Spark,4), (Dataneb,5)
)
```

- **Spark Transformations - sortByKey()**

sortByKey([*ascending*], [*numPartitions*])

When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean `ascending` argument.

```
scala> rdd.flatMap(words => words.split(" ")).distinct.map(x =>
(x,1)).sortByKey().collect
```

```
scala> rdd.flatMap(words => words.split(" ")).distinct.map(x => (x,1)).sortByKey
().collect
res10: Array[(String, Int)] = Array((Apache,1), (Dataneb,1), (Hello,1), (Spark,1
))
```

```
scala> rdd.flatMap(words => words.split(" ")).distinct.map(x =>
(x,1)).sortByKey(false).collect
```

```
scala> rdd.flatMap(words => words.split(" ")).distinct.map(x => (x,1)).sortByKey
(false).collect
res11: Array[(String, Int)] = Array((Spark,1), (Hello,1), (Dataneb,1), (Apache,1
))
```

- **Spark Transformations - join()**

join(*otherDataset*, [*numPartitions*])

When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through `leftOuterJoin`, `rightOuterJoin`, and `fullOuterJoin`.

```
scala> val rdd1 = sc.parallelize(List("Apple","Orange", "Banana",
"Grapes", "Strawberry", "Papaya")).map(words => (words,1))

scala> val rdd2 = sc.parallelize(List("Apple", "Grapes", "Peach",
"Fruits")).map(words => (words,1))

scala> rdd1.join(rdd2).collect
```

```
scala> val rdd1 = sc.parallelize(List("Apple","Orange", "Banana", "Grapes", "Str
awberry", "Papaya")).map(words => (words,1))
rdd1: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[33] at map at <
console>:24

scala> val rdd2 = sc.parallelize(List("Apple", "Grapes", "Peach", "Fruits")).map
(words => (words,1))
rdd2: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[35] at map at <
console>:24

scala> rdd1.join(rdd2).collect
res12: Array[(String, (Int, Int))] = Array((Grapes,(1,1)), (Apple,(1,1)))
```

```
scala> rdd1.rightOuterJoin(rdd2).collect
```

```
scala> rdd1.rightOuterJoin(rdd2).collect
res13: Array[(String, (Option[Int], Int))] = Array((Grapes,(Some(1),1)), (Apple,
(Some(1),1)), (Fruits,(None,1)), (Peach,(None,1)))
```

```
scala> rdd1.leftOuterJoin(rdd2).collect
```

```
scala> rdd1.leftOuterJoin(rdd2).collect
res15: Array[(String, (Int, Option[Int]))] = Array((Papaya,(1,None)), (Grapes,(1
,Some(1))), (Orange,(1,None)), (Apple,(1,Some(1))), (Strawberry,(1,None)), (Bana
na,(1,None)))
```

```
scala> rdd1.fullOuterJoin(rdd2).collect
```

```
scala> rdd1.fullOuterJoin(rdd2).collect
res16: Array[(String, (Option[Int], Option[Int]))] = Array((Papaya,(Some(1),None
)), (Grapes,(Some(1),Some(1))), (Orange,(Some(1),None)), (Apple,(Some(1),Some(1)
)), (Fruits,(None,Some(1))), (Peach,(None,Some(1))), (Strawberry,(Some(1),None))
, (Banana,(Some(1),None)))
```

- **Spark Transformations - cartesian()**

cartesian(*otherDataset*)

When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).

```
scala> val rdd1 = sc.parallelize(List("Apple","Orange", "Banana",
"Grapes", "Strawberry", "Papaya"))

scala> val rdd2 = sc.parallelize(List("Apple", "Grapes", "Peach",
"Fruits"))

scala> rdd1.cartesian(rdd2).collect
```

```
scala> val rdd1 = sc.parallelize(List("Apple","Orange", "Banana", "Grapes", "Str
awberry", "Papaya"))
rdd1: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[52] at paralleliz
e at <console>:24

scala> val rdd2 = sc.parallelize(List("Apple", "Grapes", "Peach", "Fruits"))
rdd2: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[53] at paralleliz
e at <console>:24

scala> rdd1.cartesian(rdd2).collect
res17: Array[(String, String)] = Array((Apple,Apple), (Apple,Grapes), (Orange,Ap
ple), (Orange,Grapes), (Banana,Apple), (Banana,Grapes), (Apple,Peach), (Apple,Fr
uits), (Orange,Peach), (Orange,Fruits), (Banana,Peach), (Banana,Fruits), (Grapes
,Apple), (Grapes,Grapes), (Strawberry,Apple), (Strawberry,Grapes), (Papaya,Apple
), (Papaya,Grapes), (Grapes,Peach), (Grapes,Fruits), (Strawberry,Peach), (Strawb
erry,Fruits), (Papaya,Peach), (Papaya,Fruits))
```

- **Spark Transformations - coalesce()**

coalesce(*numPartitions*)

Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset.

```
scala> val distData = sc.parallelize(1 to 16, 4)

// current partition size
scala> distData.partitions.size
```

```
scala> val distData = sc.parallelize(1 to 16, 4)
distData: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[55] at paralleli
ze at <console>:24

scala> distData.partitions.size
res18: Int = 4
```

// checking data across each partition

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 0)
iter else Iterator()).collect
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 0) iter els
e Iterator()).collect
res19: Array[Int] = Array(1, 2, 3, 4)
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 1)
iter else Iterator()).collect
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 1) iter els
e Iterator()).collect
res20: Array[Int] = Array(5, 6, 7, 8)
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 2)
iter else Iterator()).collect
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 2) iter els
e Iterator()).collect
res21: Array[Int] = Array(9, 10, 11, 12)
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 3)
iter else Iterator()).collect
```

```
scala> distData.mapPartitionsWithIndex((index, iter) => if (index == 3) iter els
e Iterator()).collect
res22: Array[Int] = Array(13, 14, 15, 16)
```

// decreasing partitions to 2

```
scala> val coalData = distData.coalesce(2)
```

// see how shuffling occurred. Instead of moving all data it just moved 2 partitions.

```
scala> coalData.mapPartitionsWithIndex((index, iter) => if (index == 0)
iter else Iterator()).collect
```

```
scala> coalData.mapPartitionsWithIndex((index, iter) => if (index == 0) iter els
e Iterator()).collect
res23: Array[Int] = Array(1, 2, 3, 4, 5, 6, 7, 8)
```

```
scala> coalData.mapPartitionsWithIndex((index, iter) => if (index == 1)
iter else Iterator()).collect
```

```
scala> coalData.mapPartitionsWithIndex((index, iter) => if (index == 1) iter els
e Iterator()).collect
res24: Array[Int] = Array(9, 10, 11, 12, 13, 14, 15, 16)
```

- **Spark Transformations - repartition()**

repartition(*numPartitions*) Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.

```
scala> val repartData = distData.repartition(2)
```

// checking data across each partition

```
scala> repartData.mapPartitionsWithIndex((index, iter) => if (index == 0) iter else Iterator()).collect
```

```
scala> val repartData = distData.repartition(2)
repartData: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[66] at repartition at <console>:25

scala> repartData.mapPartitionsWithIndex((index, iter) => if (index == 0) iter else Iterator()).collect
res25: Array[Int] = Array(1, 3, 6, 8, 9, 11, 13, 15)
```

```
scala> repartData.mapPartitionsWithIndex((index, iter) => if (index == 1) iter else Iterator()).collect
```

```
scala> repartData.mapPartitionsWithIndex((index, iter) => if (index == 1) iter else Iterator()).collect
res26: Array[Int] = Array(2, 4, 5, 7, 10, 12, 14, 16)
```

Spark Actions

When you want to work on actual dataset, you need to perform spark actions on RDDs like count, reduce, collect, first, takeSample, saveAsTextFile etc.

- *Transformations are lazy in nature, i.e., nothing happens when the code is evaluated. Meaning actual execution happens only when code is executed. RDDs are computed only when an action is applied on them. Also called as lazy evaluation. Spark evaluates the expression only when its value is needed by action.*
- *When you call an action, it actually triggers transformations to act upon RDD, dataset or dataframe. After that RDD, dataset or dataframe is calculated in memory. In short, transformations will actually occur only when you apply an action. Before that it's just line of evaluated code.*

- **Spark Actions - reduce()**

```
scala> val rdd = sc.parallelize(1 to 15).collect
```

```
scala> val rdd = sc.parallelize(1 to 15).collect
rdd: Array[Int] = Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15)
```

```
scala> val rdd = sc.parallelize(1 to 15).reduce(_ + _)
rdd: Int = 120
```

```
scala> val rdd = sc.parallelize(Array("Hello", "Dataneb", "Spark")).reduce(_ + _)
rdd: String = HelloDatanebSpark
```

```
scala> val rdd = sc.parallelize(Array("Hello", "Dataneb", "Spark")).map(x =>(x, x.length)).flatMap(l=> List(l._2)).collect

scala> rdd.reduce(_ + _)

scala> rdd.reduce((x, y)=>x+y)
```

```
scala> val rdd = sc.parallelize(Array("Hello", "Dataneb", "Spark")).map(x =>(x, x.length)).flatMap(l=> List(l._2)).collect
rdd: Array[Int] = Array(5, 7, 5)

scala>  rdd.reduce(_ + _)
res30: Int = 17

scala>  rdd.reduce((x,y)=>x+y)
res31: Int = 17
```

```
scala> val rdd = sc.parallelize(Array("Hello", "Dataneb", "Spark")).map(x =>(x, x.length)).flatMap(l=> List(l._1)).collect
rdd: Array[String] = Array(Hello, Dataneb, Spark)

scala>  rdd.reduce(_ + _)
res32: String = HelloDatanebSpark

scala>  rdd.reduce((x,y)=>x+y)
res33: String = HelloDatanebSpark
```

- **Spark Actions - collect(), count(), first(), take()**

```
scala> sc.parallelize(1 to 20, 4).collect
res34: Array[Int] = Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20)
```

//Counts the number of elements

```
scala> sc.parallelize(1 to 20, 4).count
res35: Long = 20
```

//First returns the first element

```
scala> sc.parallelize(1 to 20, 4).first
res36: Int = 1
```

//Take returns the number of elements you pass as argument

```
scala> sc.parallelize(1 to 20, 4).take(5)
```

```
scala> sc.parallelize(1 to 20, 4).take(5)
res37: Array[Int] = Array(1, 2, 3, 4, 5)
```

- **Spark Actions - takeSample()**

It returns the random sample of size n. Boolean input is for with or without replacement. For example

```
scala> sc.parallelize(1 to 20, 4).takeSample(false,4)
```

```
scala> sc.parallelize(1 to 20, 4).takeSample(false,4)
res0: Array[Int] = Array(9, 6, 20, 15)
```

```
scala> sc.parallelize(1 to 20, 4).takeSample(false,4)
res1: Array[Int] = Array(14, 2, 13, 1)
```

```
scala> sc.parallelize(1 to 20, 4).takeSample(true,4)
```

```
scala> sc.parallelize(1 to 20, 4).takeSample(true,4)
res2: Array[Int] = Array(13, 20, 13, 5)
```

- **Spark Actions - takeOrdered()**

It returns the elements in ordered fashion. For example,

```
scala> sc.parallelize(1 to 20, 4).takeOrdered(7)
```

//Just opposite to top() action

```
scala> sc.parallelize(1 to 20, 4).top(7)
```

```
scala> sc.parallelize(1 to 20, 4).takeOrdered(7)
res3: Array[Int] = Array(1, 2, 3, 4, 5, 6, 7)

scala> sc.parallelize(1 to 20, 4).top(7)
res4: Array[Int] = Array(20, 19, 18, 17, 16, 15, 14)
```

- **Spark Actions - countByKey()**

```
scala>sc.parallelize(Array("Apple","Banana","Grapes","Oranges","Grapes","
Banana")).map(k=>(k,1)).countByKey()
```

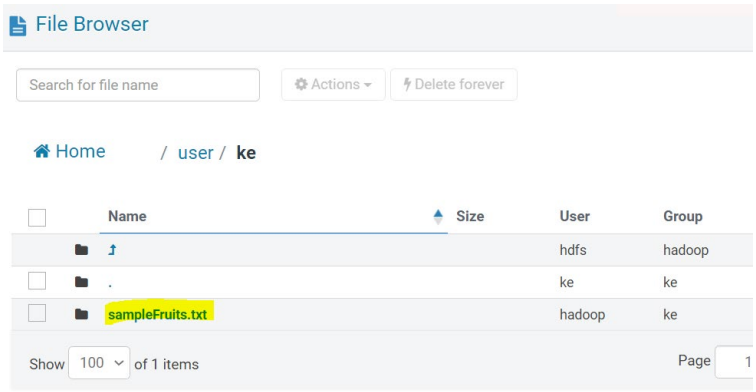
```
scala> sc.parallelize(Array("Apple","Banana","Grapes","Oranges","Grapes","Banana
")).map(k=>(k,1)).countByKey()
res5: scala.collection.Map[String,Long] = Map(Grapes -> 2, Apple -> 1, Oranges -
> 1, Banana -> 2)
```

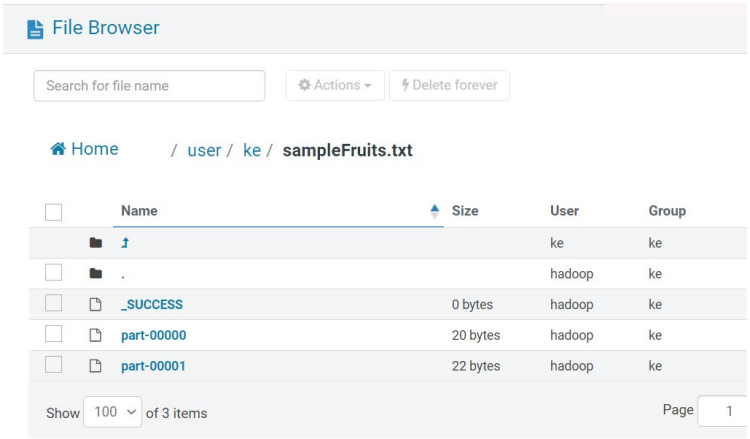
- **Spark Actions - saveAsTextFile()**

It saves the dataset as text files in local directory or HDFS etc. You can reduce the number of files by coalesce transformation.

```
scala>sc.parallelize(Array("Apple","Banana","Grapes","Oranges","Grapes","Banana")).saveAsTextFile("sampleFruits.txt")
```

```
scala> sc.parallelize(Array("Apple","Banana","Grapes","Oranges","Grapes","Banana")).saveAsTextFile("hdfs:///user/ke/sampleFruits.txt")
```

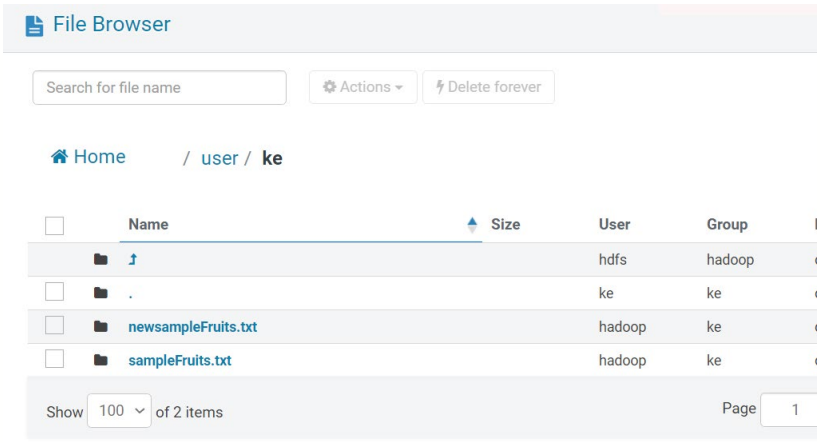


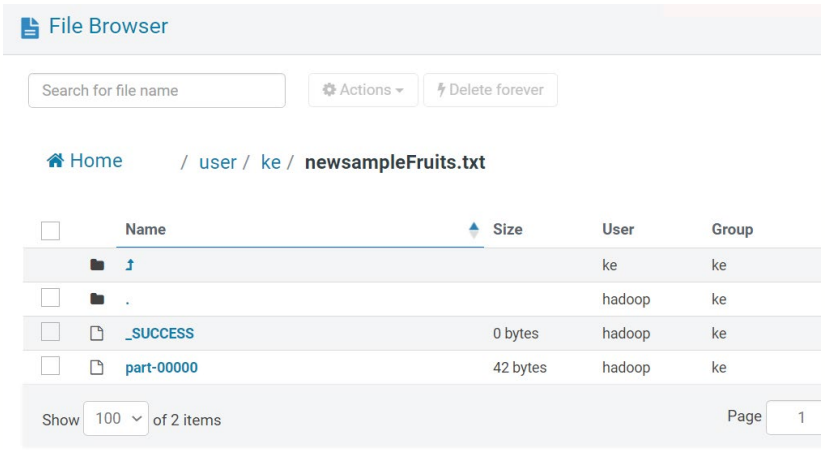


// Just one partition file with coalesce

```
scala>sc.parallelize(Array("Apple","Banana","Grapes","Oranges","Grapes","Banana")).coalesce(1).saveAsTextFile("newsampleFruits.txt")
```

```
scala> sc.parallelize(Array("Apple","Banana","Grapes","Oranges","Grapes","Banana")).coalesce(1).saveAsTextFile("hdfs:///user/ke/newsampleFruits.txt")
```





- **Spark Actions - saveAsObjectFile()**

It is generally used when you want to carry out some operation on output for each element. Like loading each element into database.

```
scala> sc.parallelize(List(1,2)).saveAsObjectFile("/Users/Rajput/sample")
```

- **Spark Actions - foreach()**

```
scala> sc.parallelize("Hello").collect
```

```
scala> sc.parallelize("Hello").collect
res11: Array[Char] = Array(H, e, l, l, o)
```

The RDD.foreach method in Spark runs on the cluster so each worker which contains these records is running the operations in foreach. I.e. your code is running, but they are printing out on the Spark workers stdout, not in the driver/your shell session.

```
scala> sc.parallelize("Hello").foreach(x=>println(x))
```

```
scala> sc.parallelize("Hello").foreach(x=>println(x))

scala>
```

```
scala> sc.parallelize("Hello").collect.foreach(x=>println(x))
H
e
l
l
o
```

scala> sc.textFile("/3littlepigs.txt").flatMap(_.split(" ")).map((_,1)).reduceByKey(_+_).saveAsTextFile("3littlepigs-wordcount")

You can read in a file from HDFS (e.g., /3littlepigs.txt), count every word, and save the output to HDFS (e.g., 3littlepigs-wordcount). Note since the output directory is not specified, the output will be saved in /user/hadoop by default)

```
scala> sc.textFile("/3littlepigs.txt").flatMap(_.split(" ")).map((_,1)).reduceBy
Key(_+_).saveAsTextFile("3littlepigs-wordcount")
```

More reading

- **What is the formal difference in Scala between braces and parentheses, and when should they be used?**